

Performance Testing

Load tests and measured results for CrossFX (Track 4). The brief requires six metrics; all six are reported below, with the method used for each and the bottleneck the numbers identify.

Contents

File	Purpose
<code>seed_users.py</code>	Creates synthetic senders — registered, KYC-approved, each with a registered recipient and a beneficiary
<code>locustfile.py</code>	The HTTP load test: quote, cash-in confirmation, wallet read
<code>measure_settlement.py</code>	Queue depth/backlog and settlement latency, which the HTTP test cannot see
<code>results/</code>	Raw Locust CSVs and captured output (gitignored)

Why the measurement is split in two

The API hands a remittance to a message queue and returns. Everything after that happens in a separate process, against the XRP Ledger Testnet, at a speed the API does not control. Timing settlement through an HTTP client would measure the wrong thing, and driving it under load would exhaust the send pool's UCTUSD liquidity — which is finite and distributed by the lecturer — within seconds.

So `locustfile.py` measures the request path and `measure_settlement.py` measures the settlement path. Keeping them apart is not a limitation of the test; it is the same boundary the architecture is built around (spec §7.2).

Running it

```
cd performance-testing
python -m venv .venv
./venv/bin/pip install locust redis

# the API must be running, with an admin account created
./venv/bin/python seed_users.py --count 40

./venv/bin/locust -f locustfile.py --host http://127.0.0.1:8000 \
  --headless -u 40 -r 10 -t 60s --csv results/api-load

./venv/bin/python measure_settlement.py
```

Results

Environment. Apple Silicon laptop, macOS 24.6. One `uvicorn` process (single worker), SQLite, Redis 8 local, `EXCHANGE_RATE_SOURCE=mock`. One settlement worker process. XRPL Testnet over the public JSON-RPC endpoint. 40 concurrent synthetic senders, 60-second runs.

1. API response times

Endpoint	Requests	Median	p95	p99	Max
<code>POST /remittances/quote</code>	2,440	8 ms	23 ms	59 ms	140 ms
<code>POST /remittances/{id}/confirm-cash-in</code>	1,616	10 ms	21 ms	35 ms	120 ms
<code>GET /wallet/balance</code>	804	4 ms	11 ms	20 ms	61 ms
<code>POST /auth/login</code>	40	530 ms	660 ms	680 ms	680 ms
Aggregated	4,900	8 ms	22 ms	89 ms	678 ms

Quoting at a median of 8 ms confirms the design intent: it is fee arithmetic, a mock FX rate and one INSERT, with no outbound network call. Had the FX source been `api` rather than `mock`, this endpoint would have been measuring a third party's uptime instead of CrossFX.

Login is two orders of magnitude slower than everything else, by design. It is the only endpoint that runs bcrypt, which is deliberately expensive to make offline password cracking impractical. A 530 ms login is a security control working, not a defect — but it means login should never sit on a hot path, and a real deployment would want more API workers to absorb the concurrency it costs.

2. Requests processed per second

82–84 req/s sustained across two 60-second runs (4,900 and 4,981 requests), from a single `uvicorn` worker with no failures. This is a single-process figure: `uvicorn` was run without `--workers`, so it is a per-core number that scales roughly linearly with workers.

3. Message-queue throughput

Measure	Value
Messages published	1,616 in 60 s
Publish rate	~27 messages/s
Publish cost on the request path	~0 ms
Consumption, single worker	0.070 messages/s

The publish side is free. Confirming cash-in had a median of 10 ms with Redis down (taking the "queue unreachable" branch) and 10 ms with Redis up — the Redis `XADD` is not measurable against the surrounding database work.

The consumption side is four orders of magnitude slower, and the reason is entirely the XRPL round trip below, not Redis.

4. UCTUSD transaction processing time

Measured over 22 real submissions to XRPL Testnet, in two runs: 12 against an **unfunded** send pool (every one rejected `tecPATH_DRY`) and 10 after the pool was funded with UCTUSD (every one `tesSUCCESS`). Timed from the worker claiming a message to recording the ledger's result.

Measure	Successful (n=10)	Rejected (n=12)
Min	10.95 s	11.25 s
Median	14.98 s	14.83 s
p95	15.28 s	15.14 s
Max	15.97 s	15.85 s
Mean	13.46 s	14.39 s
HTTP calls to XRPL per settlement	8.1	8.8
Single-worker throughput	0.074 /s ($\approx 4.4/\text{min}$)	0.070 /s

A rejected transaction costs the same as a successful one — 14.83 s against 14.98 s. Both reach a validated ledger; only the result code differs. That matters operationally: a corridor with a liquidity problem does not fail fast, it fails at full price, so failures consume worker capacity exactly as successes do.

The ~8 calls per settlement are `submit_and_wait` submitting once and then polling until the transaction appears in a validated ledger. The XRPL closes a ledger every 3–5 seconds, so a validated result is inherently a multi-second wait — a property of the ledger, not of CrossFX.

End-to-end latency under a burst

The figures above are the worker's own processing time. What a *sender* experiences also includes waiting behind other remittances in the queue. All 10 funded remittances were confirmed at once and drained by a single worker:

Measure	cash-in confirmed → settled
Min	44.6 s
Median	96.2 s
p95	162.9 s
Max	162.9 s

The last remittance in a burst of 10 waits ~2.7 minutes. This is queue depth × service time, not slowness — and it is precisely the figure that a second worker would halve.

5. Transaction success and failure rates

	Result
HTTP requests	9,881 across both load runs, 0 failures (0.00%)
Limit refusals (403)	80 — expected, see below
Settlements, funded pool	10 attempted, 10 succeeded (100%)
Settlements, unfunded pool	12 attempted, 12 failed <code>tecPATH_DRY</code> , all recorded as <code>FAILED</code>

Both halves of the brief's XRPL requirements are therefore demonstrated against the live ledger rather than a mock: successful submission and validation, and failed-transaction handling.

The 12 failures were a liquidity condition, not a defect. `tecPATH_DRY` means the send pool held no UCTUSD to pay. Every failure was caught, recorded against the remittance as `FAILED` with its XRPL result code, and surfaced on the recipient's wallet as a failed entry **with no credit**. After the pool was funded with 1,000 UCTUSD, the same code path settled 10 of 10.

Reconciliation

The invariant in spec §9.6 — that the payout pool's on-chain balance equals the sum of every user's UCTUSD claim in the internal ledger — was checked after the successful run and holds exactly:

	UCTUSD
Payout pool, on-chain	91.00872
Sum of user ledger claims	91.00872
Send pool, on-chain	908.99128

Ten remittances of 9.100872 UCTUSD each moved from the send pool to the payout pool on-chain, and exactly that much was credited to recipients internally. No drift, which is the evidence that pooled custody is being accounted correctly.

The 80 limit refusals are the limit model working. Each synthetic sender sends R50 at a time against a R3,000 daily ceiling, so each is refused after exactly 60 quotes — 40 users × 60 = 2,400 quotes, then 40 refusals. The counts matched exactly on both runs. These are excluded from the failure rate because a business rule declining a well-formed request is not the API failing.

6. System behaviour under concurrent use

40 concurrent senders produced no errors, no database lock contention, and no latency collapse: the p95 stayed at 22 ms while the median stayed at 8 ms. Ramping from 0 to 40 users over 4 seconds produced no error spike.

Concurrency safety is structural rather than incidental. Each synthetic sender has its own recipient, so concurrent settlements touch different `ledger_balances` rows; where they would collide, `Ledger` takes `SELECT ... FOR UPDATE` on the balance row, and the settlement worker claims each remittance with a compare-and-swap so a redelivered message cannot double-credit.

Bottleneck analysis

The XRPL round trip dominates everything — and the queue is why that is survivable

Stage	Median
Confirm cash-in (API request path)	10 ms
Settle on XRPL (worker path)	14,980 ms

The on-chain leg is **~1,500× slower than the request that triggers it**. Had settlement been synchronous, every sender would wait ~15 seconds for a response and the API's throughput ceiling would be 0.07 requests/s per worker instead of 82. The measured gap is the strongest quantitative argument in the project for the architecture the brief mandates: the queue is not an optimisation, it is what makes the system usable at all.

Settlement throughput scales with workers, not with tuning

One worker settles 4.4 remittances per minute, and essentially all of its 15 seconds are spent waiting on the network — the measured gap between a successful and a rejected settlement is 0.15 s, so almost none of the time is CrossFX's own work. It is purely I/O-bound, so N workers sharing the Redis consumer group should give close to N times the throughput until XRPL rate limits bind. Nothing in the worker needs optimising — it needs replicating. This is why the queue uses a consumer group rather than a plain `XREAD`.

The burst figures make the case concretely: the tenth remittance in a batch of 10 waited 162.9 s, of which only ~15 s was its own settlement. Everything else was queue wait that a second worker would halve.

bcrypt is the slowest thing in the API, deliberately

At 530 ms median, login costs as much as 65 quotes. It is the correct trade-off, but it means a burst of logins — the first minute of a demo, or of a marketing campaign — needs more API workers than steady-state traffic does.

What was not reached

SQLite's single-writer limit never manifested at 82 req/s, so the database was not the bottleneck at this scale. A deployment on Postgres (which the models and migrations already support unchanged, spec §7.4) would be needed to find where it does bind. This is the most obvious next test, and it is untested here rather than assumed away.

Caveats

- **Settlement samples are small** — 10 successful and 12 rejected. Enough to establish the order of magnitude and the ledger-close floor, not enough for a meaningful tail. The narrow spread (10.95–15.97 s across all 22) suggests the distribution is dominated by the ledger close interval rather than by variance in CrossFX.

- **Testnet is not mainnet.** Validation timing on a public test network is indicative, not a production SLA.
- Single-machine test: client, API, worker, Redis and database all shared one laptop, so the client competed with the server for CPU. The true API ceiling is therefore somewhat higher than 84 req/s.
- One uvicorn worker and one settlement worker throughout. Neither multi-worker scaling claim above was measured directly.